Instance Search with Deep and Handcrafted Features

CS5187 Vision and Image — Assignment 1

1 Task and data

Given a query image with one or more annotated bounding boxes, the task is to rank all 28,493 gallery images by how likely they contain the same instance, and to localise that instance in the retrieved images. Two independent matching methods are implemented: a deep region-feature pipeline (method A) and a handcrafted SIFT/bag-of-words pipeline (method B). Each produces a complete rank list, so the better of the two can be used.

Property	Value
Gallery images	28,493 (ids 0–28,492, contiguous)
Query images	50, each with an annotation file
Instances per query	1 for 40 queries, 2 for 10 queries
Bounding-box area	2.8% – 88.1% of the image (median 16.9%)
Bounding-box aspect w/h	0.30 – 4.01 (median 0.99)

Table 1 Dataset statistics, measured by the scan stage.

Two rows of Table 1 shape the design. The instance covers only about 17% of the query image on median and less in the gallery, so matching a whole image against a whole image would dilute it; both methods therefore compare the query region against regions of the gallery image. Box aspect ratios span a factor of more than ten, so neither method may assume square windows.

2 Method A: ResNet-50 region features

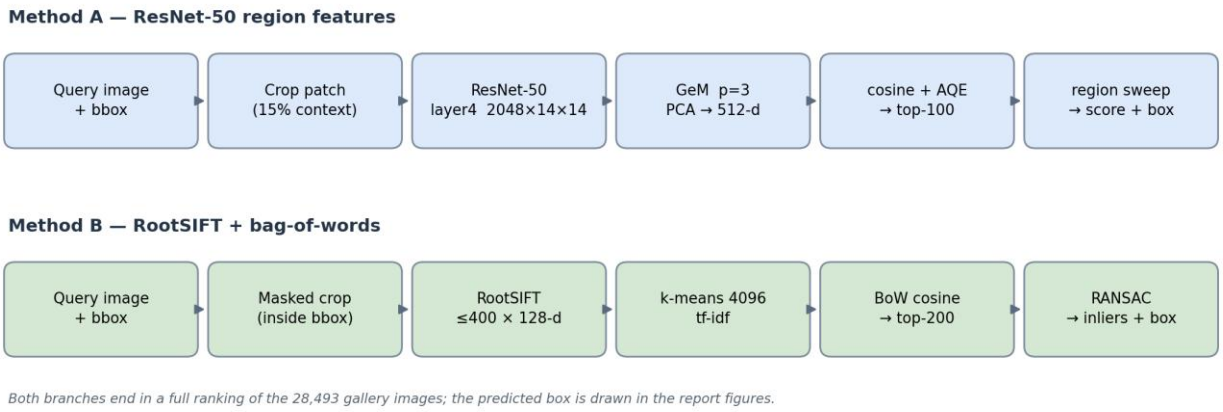


Figure 1 The two pipelines. Blue: method A (deep). Green: method B (handcrafted).

Method A follows a retrieve-then-re-rank design. A ResNet-50 pre-trained on ImageNet is truncated after layer4, so a gallery image resized to a 448-pixel short side yields a 2048×14×18 feature map. One forward pass per image produces both a global descriptor (generalised mean pooling, p = 3, then PCA-whitening to 512 dimensions) and the map that the re-ranking stage later re-uses.

Stage 1 compares the whitened query descriptor against the 28,493×512 gallery matrix with a cosine score, after average query expansion (the query is replaced by a weighted mean of itself and its five nearest neighbours). Stage 2 reloads the 100 best candidates and enumerates windows over their cached feature maps at four scales (0.35–0.8 of the map height) and three aspect multipliers (0.7, 1.0, 1.4 of the query box aspect). Because each candidate is already in memory and the window descriptors are obtained by pooling the map, this sweep costs no extra forward passes: per-channel integral images make the sum over any rectangle four lookups. The best-scoring window gives the region similarity, and its box is refined by averaging the eight strongest overlapping windows with softmax weights, which removes most of the one-cell jitter of a raw arg-max. The two scores are fused as 0.35·global + 0.65·region on raw cosine values; queries with two boxes are searched independently and merged by taking the higher fused score per gallery image.

3 Method B: RootSIFT + bag-of-words + RANSAC

Method B is intentionally built only from course material. SIFT keypoints are detected on the greyscale image (at most 400 per image) and each 128-d descriptor is converted to RootSIFT by L1-normalising and taking the square root. For a query, the detector runs inside a mask covering the annotated box, which suppresses background descriptors before they can pollute the histogram.

A vocabulary of 4,096 visual words is fitted with k-means on 400,000 sampled descriptors. Each image is encoded as a term-frequency histogram and weighted by inverse document frequency, then L2-normalised, so cosine similarity between two histograms is a dot product. The 200 best candidates go through geometric verification: Lowe's ratio test (0.75) followed by a RANSAC homography with a 5-pixel threshold. The homography is used twice — to count inliers (the verification score) and to project the query box into the gallery image (the predicted box). Verified images are ranked by inlier count and placed ahead of the unverified remainder, which keeps the BoW order for the tail.

4 Implementation and cost

Both pipelines share one data layer (identifier enumeration, bounding-box parsing, image I/O) and one entry point: python main.py {deep,hand} --stage ... . Everything runs on a single machine with an RTX 4060 Laptop GPU (PyTorch 2.11, CUDA 12.8); SIFT extraction uses 12 worker processes. Gallery features are cached, and the deep stage is chunked so an interrupted extraction resumes instead of restarting.

Stage	Method A	Method B
Feature extraction	14.0 min (33.9 img/s)	8.3 min (57.2 img/s, 12 workers)
Index construction	—	6.0 min (k-means + BoW + tf-idf)
All 50 queries	6.4 min (6–13 s each)	0.2 min (0.1–1.1 s each)
Cache size	58 MB (28,493 × 512)	9.3 M non-zeros (28,493 × 4,096)

Table 2 Measured cost. Method B is an order of magnitude faster at query time because its index is a sparse matrix, while method A pays for one forward pass per candidate.

5 Results

Both rank lists contain 50 rows with all 28,493 identifiers in descending order and pass the format validator (row count, prefix order, ids per row, duplicates, range). The figures below show the top-10 with predicted boxes in yellow and query boxes in green; the bottom-left cell of each figure shows the crops the encoder actually receives.

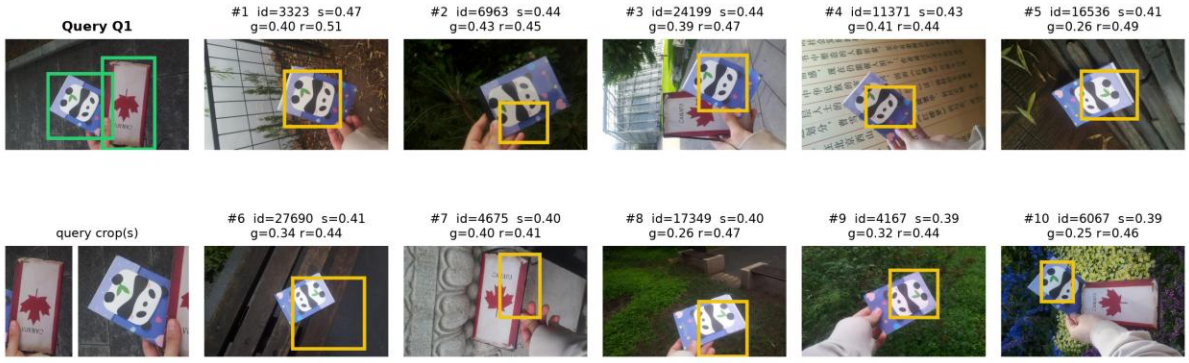


Figure 2 Query 1, method A. All ten results contain one of the two instances (a panda card or a Canada notebook); the boxes are close but slightly loose.

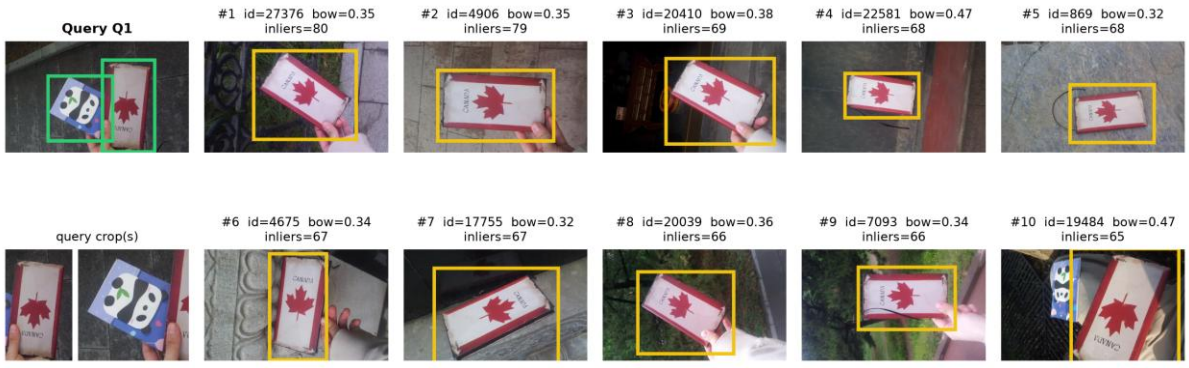


Figure 3 Query 1, method B. Every result is the Canada notebook and the homography box hugs the object more tightly than method A's window response.

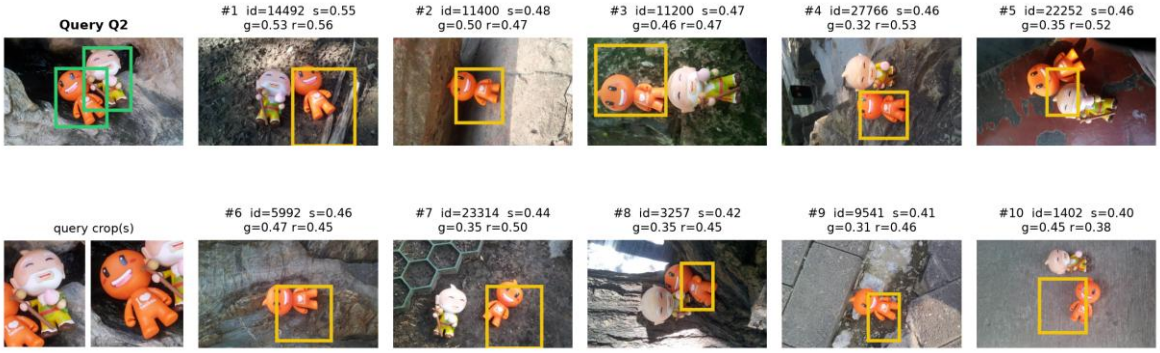


Figure 4 Query 2, method A.

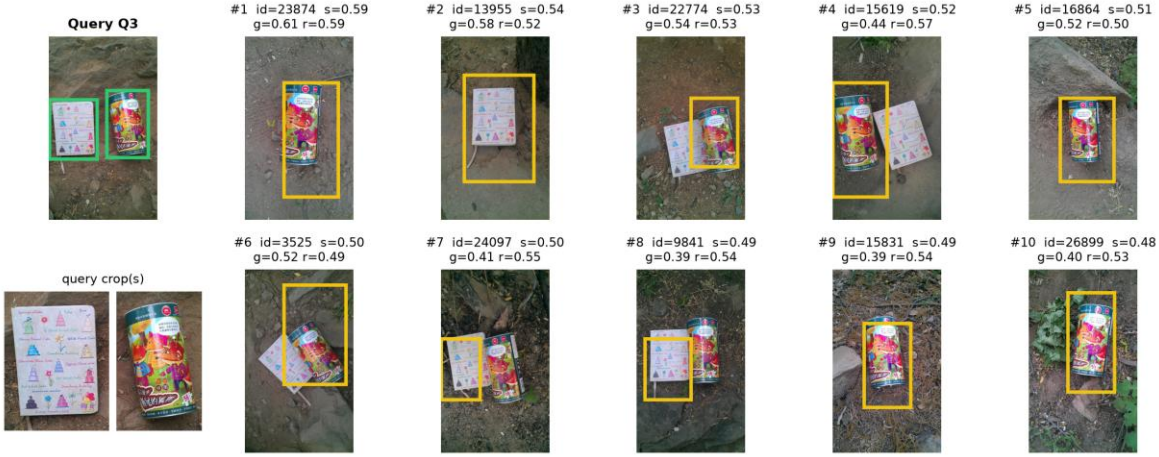


Figure 5 Query 3, method A.

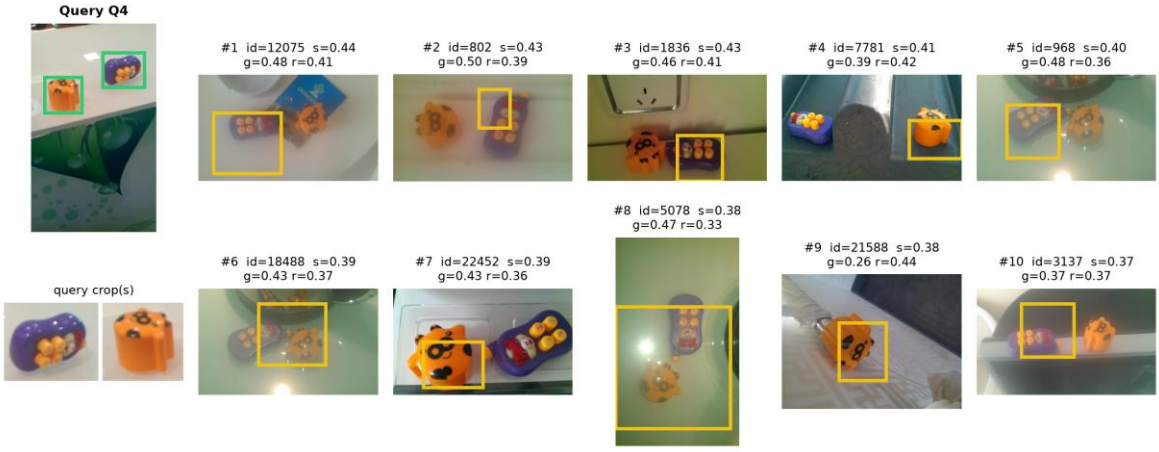


Figure 6 Query 4, method A.

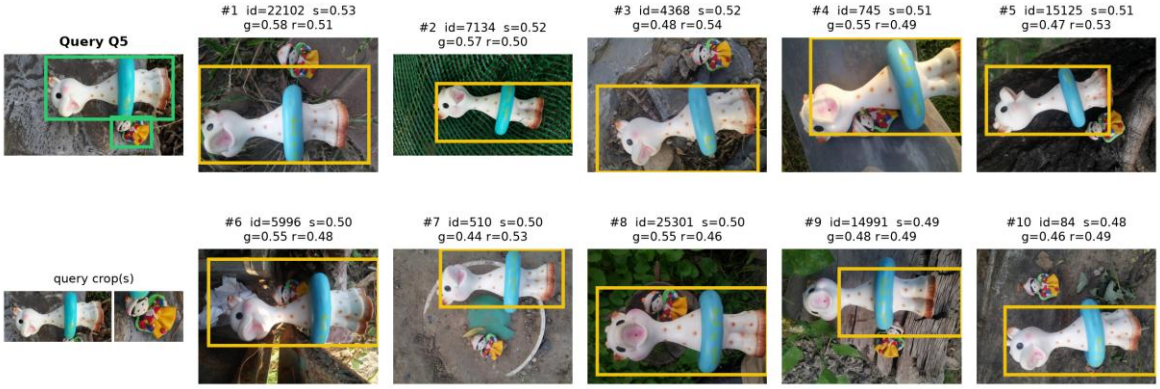


Figure 7 Query 5, method A.

Across the 50 queries the fused top-1 score of method A has median 0.51 (range 0.29–0.79), and the median margin between rank 1 and rank 10 is 0.10; 16 queries fall below 0.45, which are the crowded scenes where the instance is small. Method B verifies a median of 28 inliers for its rank-1 result, but 15 queries have no verified candidate at all, and on average only 4.8 of its top-10 images pass RANSAC.

6 Discussion

The two methods fail in different ways, which is the useful part of having both.

- Method A is the better retriever: it tolerates viewpoint, scale and partial occlusion, and it covers both instances of a two-instance query. Its weakness is localisation — feature cells are 32 pixels, so a box can be a cell or two off, and for objects smaller than ~15% of the image height the similarity surface is flat enough for the arg-max to slide onto background.
- Method B is the better localiser: a homography either agrees with the object or fails, so its boxes are tight and its top ranks are rarely wrong when inliers are high. Its weakness is recall — repetitive texture (packaging, printed patterns) produces many consistent matches on the wrong object, and a query whose crop is textureless has too few SIFT keypoints to rank anything.
- The fused design paid off twice. Scoring raw cosines instead of per-box normalised scores matters: an earlier version normalised within each query box and promoted an unrelated image to rank 1 on query 1, because it happened to be the best of a weak candidate set.

Three improvements are obvious from the failure cases: re-rank candidates at a higher resolution than 448 pixels so the grid is finer; use the verified boxes of method B to re-rank method A's head (a cheap cascade, since B answers in 0.2 minutes); and learn a projection that maps both descriptors into a shared space, which would let one score combine semantic and geometric evidence instead of fusing two heterogeneous cosines.

7 Reproducibility

The whole study regenerates from the raw data with five commands.

```
python main.py deep --stage all        # run1.txt + figures
python main.py hand --stage all        # run2.txt + figures
python main.py deep --stage validate   # format gate
```